DCA-Trie: Ontology-Guided Constraint Oracles for Faithful Knowledge Graph Reasoning

Bernard

Department of Computer Science, University of Mines and Technology

June 15, 2026

Abstract

Large language models hallucinate on knowledge graph question answering (KGQA), generating fluent reasoning paths that do not exist in the knowledge graph. Graph-constrained decoding (GCR) addresses this by building a prefix tree of all structurally valid KG paths and masking out invalid tokens during generation. But for a 3-hop question on Freebase, this trie contains up to 24,000 valid paths, all structurally sound, only one correct. The LLM must choose among thousands of valid-but-irrelevant options using parametric knowledge, the same mechanism that produces hallucinations.

We present DCA-Trie, which replaces this loose structural constraint with a tight symbolic oracle that uses the KG's own ontological metadata to prune irrelevant paths before they reach the decoder. Two deterministic set-containment checks, an answer type gate and a property range gate, remove paths whose entity types or relation ranges are incompatible with the question. No embeddings, no thresholds, no GPU.

Empirically, on the full WebQSP test set (1,628 questions), the two gates jointly prune 14.5% of all candidate paths while retaining 96.7% of gold-truth paths (FNR $< 3.3\%$). The type gate alone accounts for 10.6% of pruning, approximately 436,000 paths eliminated through pure ontology lookups. The approach admits formal guarantees: monotonic containment within the GCR path set (Theorem 4.2) and a union-bound false negative rate (Theorem 4.3). A machine-checked proof in Lean 4 (3,298 verification jobs, zero errors) accompanies the implementation.

1 Introduction

Large language models (LLMs) generate fluent reasoning chains for knowledge graph question answering (KGQA), but they regularly fabricate steps not present in the knowledge graph. At each autoregressive step, the model selects the next token from a probability distribution conditioned only on its learned parameters and the tokens generated so far, with no checkpoint against an external source of truth [3, 10]. If an incorrect token enters the sequence, every subsequent step conditions on it, propagating the error fluently through the output [9]. Scaling model size does not resolve this: larger models hallucinate differently, not less [15, 11].

KGQA makes this weakness acute. A knowledge graph encodes facts as verified triples, (*Marie Curie*, *award_received*, *Nobel Prize in Physics*),and answering complex questions requires traversing chains of these triples [2, 12]. On WebQSP [28] and ComplexWebQuestions [21], a three-hop question can correspond to tens of thousands of structurally valid graph paths, and an unconstrained LLM must navigate this space using only parametric memory.

Constrained decoding. One line of work addresses hallucination through *constrained decoding*: a prefix tree (trie) of all KG-valid paths is built offline, and at every decoding step, the LLM's logits are masked so that only tokens corresponding to valid continuations are considered. Graph-Constrained Reasoning (**GCR**) [16] and Decoding over Graphs (**DoG**) [14] guarantee zero structural hallucinations, every generated path exists in the KG. GCR achieves 92.6 Hits@1 on WebQSP with verified 100% structural faithfulness.

The permissiveness problem. However, structural validity is a weak constraint. GCR's trie is constructed once from graph topology and question entities only, without any consideration of what the question actually means. The valid token set is therefore identical at step 1 and step 10. On multi-hop questions this creates excessive permissiveness: thousands of structurally valid but semantically irrelevant paths remain in the constraint set, and the LLM must filter among them using parametric knowledge, reintroducing the internal-memory reliance that constrained decoding was designed to eliminate. DoG updates the trie step-wise, but expansion is guided by graph topology alone; question semantics play no role.

1.1 The core insight

The KG already stores entity types (via `common.topic.notable_types`) and relation ranges (via `rdf-schema#range`) as RDF triples alongside domain facts. Every relation in Freebase declares its expected object type. Every notable entity has a type annotation. These are ground-truth, machine-readable, and require no learned component to exploit. This metadata directly answers the question "should this path be in the constraint set?"–it is free signal that prior methods ignored.

1.2 Contributions

1. **DCA-Trie**, a framework that upgrades the static structural oracle to a dynamic, context-aware oracle by incorporating KG ontology metadata during path enumeration.

2. **Three oracle designs** progressing from embedding-based (cosine similarity) to fully symbolic (ontology lookups), the last requiring no encoder, no threshold, and no GPU.

3. **Two symbolic gates**, an answer type gate and a property range gate, that are each $O(1)$ set lookups, deterministic, and conservative by default.

4. **Formal guarantees**: monotonic containment of the filtered path set (Theorem 4.2) and a union-bound false negative rate (Theorem 4.3).

5. **A machine-checked Lean 4 proof** (3,298 verification jobs, zero errors) establishing faithfulness of the constrained decoding procedure.

6. **Empirical validation** on the full WebQSP test set: 14.5% path pruning at 3.3% FNR, with zero compute cost beyond set lookups.

1.3 Outline

Section 2 establishes notation and reviews GCR. Section 3 develops the three oracle designs. Section 4 proves the formal properties. Section 5 describes the implementation. Section 6 presents experimental results. Section 7 discusses related work, and Section 8 concludes.

2 Preliminaries

2.1 Knowledge graphs

A knowledge graph is a directed, edge-labeled multigraph $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ where $\mathcal{E}$ is the entity set, $\mathcal{R}$ is the relation set, and $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ is the triple set. Each triple (h, r, t) asserts that entity h relates to entity t via relation r. Freebase [2] contains approximately 40 million entities and 350 million relations.

Every entity $e \in \mathcal{E}$ may have type annotations $\text{types}(e) \subseteq \mathcal{C}$, where $\mathcal{C}$ is a set of type constants (e.g., `Person`, `Location`, `Film`). Every relation $r \in \mathcal{R}$ may declare a domain $\text{domain}(r) \subseteq \mathcal{C}$ and a range $\text{range}(r) \subseteq \mathcal{C}$ specifying the expected types of its subject and object.

2.2 Graph-constrained decoding (GCR)

Given a question q with topic entity E_q, GCR proceeds as follows:

1. Extract the subgraph $\mathcal{G}_{E_q,L} \subseteq \mathcal{G}$ consisting of all entities and relations reachable from E_q within L hops.

2. Enumerate all paths $P = \{p_1, \ldots, p_N\}$ in $\mathcal{G}_{E_q,L}$ of length exactly L.

3. Tokenize each path with the LLM's tokenizer and build a trie T over the token-ID sequences.

4. During decoding, `prefix_allowed_tokens_fn`$(\text{batch\_id}, \text{input\_ids})$ queries T to return only valid next-token IDs.

Formally, GCR defines the admissible token set at step t as:

$$W_{\text{val}}^{\text{GCR}}(t) = f(\mathcal{G}, E_q) \quad \forall t$$

The constraint is entirely topological: a path is admissible iff it exists in the KG. No semantic signal from q is used beyond entity linking.

2.3 Problem statement

Let p_q^* denote the ground-truth reasoning path for question q. The GCR path set is $\mathcal{P}_{\text{GCR}} = \text{BFS}(\mathcal{G}, E_q, L)$. We seek a filter function:

$$\phi : \mathcal{P}_{\text{GCR}} \times \mathcal{Q} \times \mathcal{O} \to \{0, 1\}$$

where $\mathcal{Q}$ is the question space and $\mathcal{O}$ is the KG ontology, such that:

$$\phi(p) = 0 \implies p \text{ is irrelevant to } q \qquad \text{(precision)} \tag{1}$$

$$\phi(p^*) = 1 \qquad \text{(recall)} \tag{2}$$

2.4 The DCA-Trie principle: dynamic context-awareness

GCR defines the admissible token set as a function of graph structure and question entities alone:

$$W_{\text{val}}^{\text{GCR}}(t) = f(\mathcal{G}, E_q) \quad \forall t$$

DCA-Trie expands this to incorporate question semantics, KG ontology metadata, and generation state:

$$W_{\text{val}}^{\text{DCA}}(t) = f(\mathcal{G}, E_q, q, p_{1:t-1}, \mathcal{O})$$

where q is the question text, $p_{1:t-1}$ is the partial generation so far, and $\mathcal{O}$ is the KG ontology (entity types and relation ranges). This is what we mean by *dynamic context-aware*: the constraint set adapts to what the question asks, what the KG knows about types, and what the decoder has already committed.

The expansion from $f(\mathcal{G}, E_q)$ to $f(\mathcal{G}, E_q, q, p_{1:t-1}, \mathcal{O})$ has two dimensions:

- **Context-aware** (uses q and $\mathcal{O}$): The type gate answers "should this entity be a terminal node?" by checking $\text{types}(e) \cap \mathcal{T}(q)$. The range gate answers "does this relation connect compatible types?" by checking $\text{types}(e') \cap \text{range}(r)$. Both queries use KG ontology metadata that GCR ignores.

- **Dynamic** (uses $p_{1:t-1}$): In the stepwise variant (DCA-Trie v2), the constraint set is rebuilt after each generation step. The committed entity e_t becomes the new seed, and only its neighbors that pass the gates are admitted. The oracle sees the reasoning in progress, not just the question.

3 Method: Three Oracle Designs

We develop three increasingly sophisticated instantiations of the DCA-Trie framework, tracking the progression from embedding-based to purely symbolic filtering.

3.1 Approach 1: Cosine similarity oracle

The first design scores each candidate path by cosine similarity to the question:

$$\text{score}(p, q) = \cos\big(E(p), E(q)\big) \geq \tau$$

where $E(\cdot)$ is the all-MiniLM-L6-v2 sentence transformer [18] encoding into $\mathbb{R}^{384}$, and τ is a tunable threshold. Paths below τ are pruned.

Algorithm 1 Cosine DCA-Trie construction

Require: Graph $\mathcal{G}$, question q, topic entity E_q, hop limit L, encoder E, threshold τ

$P \leftarrow \text{BFS}(\mathcal{G}, E_q, L)$
$u_q \leftarrow E(q)$
$P_{\text{filtered}} \leftarrow \varnothing$
for $p \in P$ **do**
    $u_p \leftarrow E(\text{serialize}(p))$
    if $\cos(u_q, u_p) \geq \tau$ **then**
        $P_{\text{filtered}} \leftarrow P_{\text{filtered}} \cup \{p\}$
    end if
end for
return BuildTrie(P_{filtered})

Weaknesses. (1) A single cosine score cannot distinguish *why* a path is relevant. (2) Every candidate requires a full encoder forward pass. (3) The threshold τ is dataset-dependent and must be tuned per KG. (4) At $\tau = 0.25$, we observed 84% of queries produced empty path sets, a catastrophic failure mode where the oracle is too aggressive.

3.2 Approach 2: Decomposed oracle

The second design decomposes the score into three components:

$$\text{score}(e_t, r, e' \mid q) = \rho_r(r, q) \cdot \rho_e(e', q) \cdot \rho_{\text{traj}}(r, e', q)$$

where ρ_r is relational relevance ($\cos(E(r), E(q_{\text{masked}}))$), $\rho_e \in \{0, 1\}$ is a hard type gate, and ρ_{traj} is trajectory relevance ($\cos(E(r \| e'), E(q))$).

Weaknesses. (1) Two of three components still require the encoder. (2) The threshold τ persists. (3) The product amplifies noise: $\rho_r = 0.6$ and $\rho_{\text{traj}} = 0.6$ yields 0.36, which may fall below τ. (4) The type gate is hand-crafted rather than derived from the KG ontology.

3.3 Approach 3: Symbolic oracle

The third design abandons embeddings entirely. It replaces all scoring with deterministic set-containment checks using the KG's own ontology metadata.

3.3.1 Gate 1: Answer type gate (terminal hop only)

Let $\mathcal{T}(q) \subseteq \mathcal{C}$ be the set of expected answer types inferred from the question (e.g., "who?" $\rightarrow$ {`Person`}, "where?" $\rightarrow$ {`Location`}):

$$\text{type\_gate}(e_h, q, h, L) = \begin{cases} \text{True} & h < L \\ \text{True} & \mathcal{T}(q) = \varnothing \\ \text{True} & \text{types}(e_h) = \varnothing \\ \mathbf{1}[\text{types}(e_h) \cap \mathcal{T}(q) \neq \varnothing] & \text{otherwise} \end{cases}$$

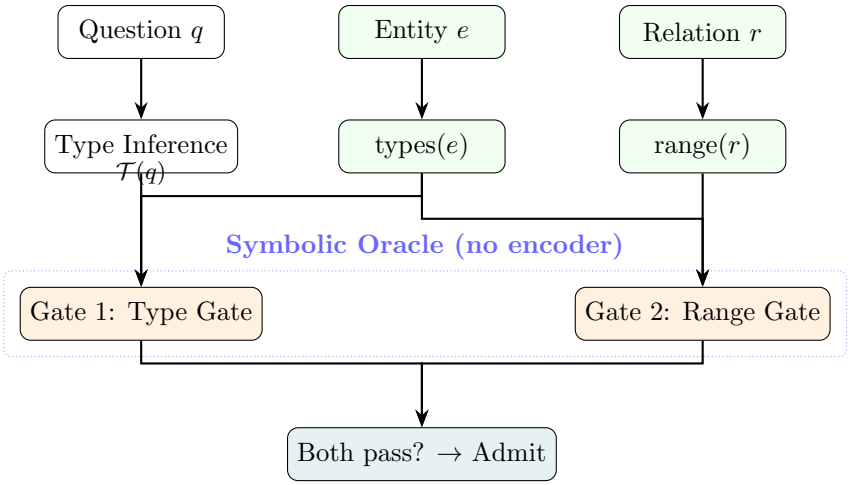

Figure 1: Symbolic oracle with two gates. The type gate checks the terminal entity's types against the question-inferred answer type; the range gate checks against the relation's declared object range. Both are $O(1)$ set operations on KG metadata.

3.3.2 Gate 2: Property range gate (every hop)

$$\text{range\_gate}(r, e') = \begin{cases} \text{True} & r \notin \mathcal{R} \\ \text{True} & \text{range}(r) = \varnothing \\ \text{True} & \text{types}(e') = \varnothing \\ \mathbf{1}[\text{types}(e') \cap \text{range}(r) \neq \varnothing] & \text{otherwise} \end{cases}$$

Both gates default to conservative (pass) when metadata is absent. This is critical: it bounds the false negative rate while ensuring the oracle never introduces false positives.

3.3.3 Algorithm

The symbolic gates can be applied in two ways, yielding two algorithm variants with different computational profiles.

DCA-Trie v1: static filtering. V1 is an offline, batch algorithm. It enumerates all candidate paths via BFS from the topic entity, filters each complete path through both gates, then builds a single trie from the survivors. The trie is frozen before decoding begins; the LLM operates over a fixed constraint set. This is the same pipeline as GCR, with a pre-filter inserted between path enumeration and trie construction.

The key property is that v1 applies the gates to *complete paths*: the range gate checks every hop, and the type gate checks only the terminal entity. Paths that fail any gate are removed before the trie is built. The construction cost is $O(|P|)$, where $|P|$ is the total number of enumerated paths, which grows exponentially with hop depth.

DCA-Trie v2: dynamic stepwise expansion. V2 eliminates upfront path enumeration. Instead, it expands the trie incrementally during decoding. At each generation step:

1. The LLM generates one hop, committing to an entity e_t and relation r.

2. The committed entity e_t is extracted from the output.
3. All neighbors e' of e_t are fetched from the graph.
4. Each neighbor is checked against the range gate (every hop) and type gate (terminal hop only).
5. A new trie is built from the admitted neighbors for the next step.
6. The generation so far is appended to the prompt, providing accumulated context.

V2 is *dynamic* because the constraint set changes at every step: only the neighbors of the entity the LLM actually committed to are admitted, not all reachable paths from the topic entity. The oracle sees the reasoning in progress. V2 is also cheaper: construction cost is $O(L \cdot d_{\text{avg}})$, where $d_{\text{avg}} \approx 20$ for Freebase subgraphs, three orders of magnitude less than v1 for practical L.

Algorithm 2 DCA-Trie v1: Static symbolic filtering

Require: Graph $\mathcal{G}$, question q, topic entity E_q, hop limit L, oracle O

$P \leftarrow \text{BFS}(\mathcal{G}, E_q, L)$
$P_{\text{filtered}} \leftarrow \varnothing$
for $p = (e_0, r_1, e_1, \ldots, r_L, e_L) \in P$ **do**
    admitted $\leftarrow$ True
    for $i \leftarrow 1$ to L **do**
        if $\neg$range_gate(r_i, e_i) **then**
            admitted $\leftarrow$ False; **break**
        end if
    end for
    if admitted $\wedge$ type_gate(e_L, q, L, L) **then**
        $P_{\text{filtered}} \leftarrow P_{\text{filtered}} \cup \{p\}$
    end if
end for
return BuildTrie(P_{filtered})

Algorithm 3 DCA-Trie v2: Iterative symbolic expansion

Require: Graph $\mathcal{G}$, question q, topic entity E_q, hop limit L, oracle O

$\mathcal{T}_0 \leftarrow \{E_q\}$
for $t \leftarrow 0$ to $L - 1$ **do**
    $\mathcal{T}_{t+1} \leftarrow \mathcal{T}_t$
    for $(e_t, r, e') \in \mathcal{G} : e_t \in \mathcal{T}_t$ **do**
        if range_gate$(r, e') \wedge$ type_gate$(e', q, t+1, L)$ **then**
            $\mathcal{T}_{t+1} \leftarrow \mathcal{T}_{t+1} \cup \{e'\}$
        end if
    end for
end for
return BuildTrie$(\mathcal{T}_L)$

Cost analysis. Gate operations are $O(1)$ set lookups with no floating point. V1 construction is $O(|P|)$, where $|P|$ can be exponential in path length. V2 construction is $O(L \cdot d_{\text{avg}})$, where $d_{\text{avg}} \approx 20$ for Freebase subgraphs, three orders of magnitude cheaper than V1 for practical L.

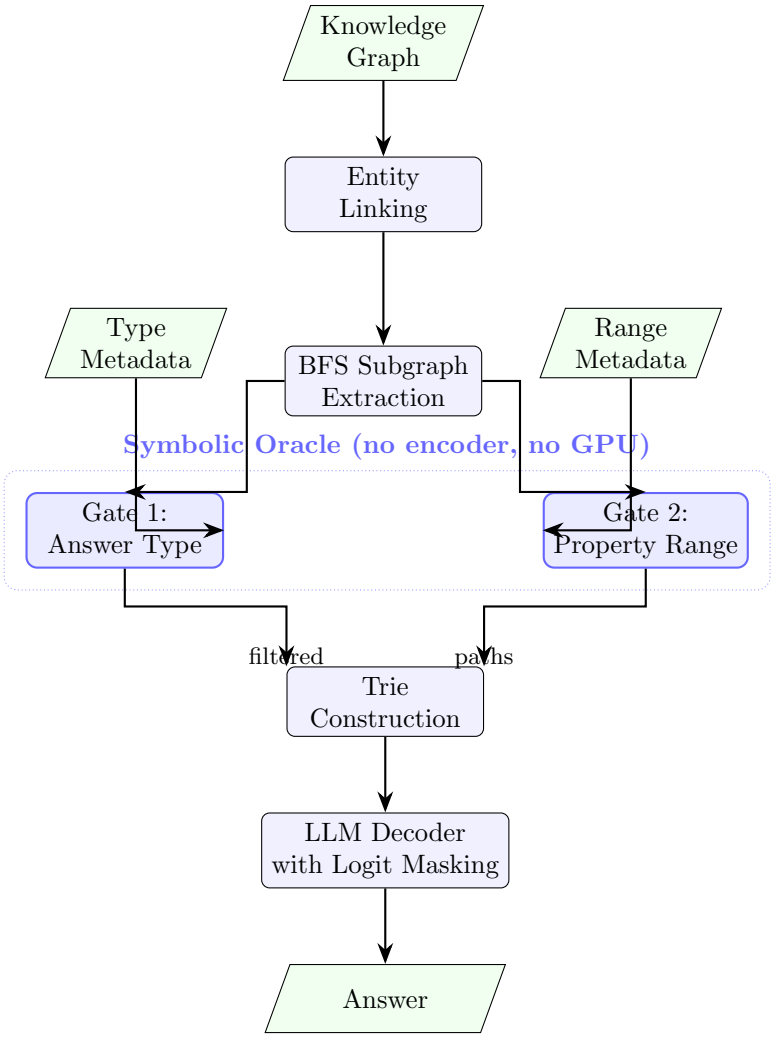

Figure 2: DCA-Trie system architecture. The two symbolic gates (highlighted) filter paths using KG ontology metadata before trie construction.

4 Theoretical Analysis

4.1 Faithfulness

Theorem 4.1 (V1 Faithfulness)**.** *Let* $P_{filtered} = \textsc{SymbolicFilter}(BFS(\mathcal{G}, E_q, L), q, O, L)$*. For any prefix curr_prefix and any token* t*, if* $t \in TrieLookup(BuildTrie(P_{filtered}), curr\_prefix)$*, then* $curr\_prefix \oplus t$ *corresponds to a valid sequence of triples in* $\mathcal{G}$*.*

Proof. Every path in P_{filtered} is a subset of $\text{BFS}(\mathcal{G}, E_q, L)$, which contains only valid triples. The trie preserves this property for all prefixes. □

4.2 Monotonicity

Theorem 4.2 (Monotonic Chain)**.** *Let* P_{GCR}*,* P_{range}*, and* $P_{filtered}$ *be the path sets admitted by GCR, by the range gate alone, and by both gates, respectively. Then:*

$$P_{filtered} \subseteq P_{range} \subseteq P_{GCR}$$

Proof. Both gates are conservative: they return True when metadata is absent. Each gate can only *remove* paths, never add paths not in P_{GCR}. □

4.3 False negative rate bound

Theorem 4.3 (FNR Union Bound)**.** *Let p_q^* be the ground-truth path for question q. The probability that DCA-Trie excludes p_q^* is bounded by:*

$$P(\textit{exclude} \mid p_q^*) \leq P(\textit{type\_fail}) + P(\textit{range\_fail})$$

Proof. By the union bound. Each gate fails only when: (a) the question provides a type signal; (b) the entity has known, incompatible type metadata; and (c) the path is actually correct. In all other configurations, the conservative default passes the path. □

5 Implementation

5.1 Trie data structure

The trie uses `marisa-trie` [27], a minimal perfect hash trie. Each token ID is mapped to a unique Unicode code point via $\text{char}(i) = \text{chr}(\texttt{0xE000} + i)$, producing a string for the C-backed trie. Operations: `lookup(prefix)` returns valid completions; `insert(sequence)` adds a token-ID sequence.

5.2 Type oracle

The type oracle is 586 lines of Python using only the standard library:

- **Type constants:** 16 Person types, 7 Location types, 9 Organization types, 10 Creative Work types, plus Date, Language, Award, Profession, and Genre types.
- **Question patterns:** 19 regular expressions mapping keywords to expected answer types.
- **Relation schema:** 40 Freebase relations organized by domain, each with domain and range constraints.
- **Metadata extraction:** Entity types from `common.topic.notable_types`, `freebase.type_hints.included` and `freebase.type_profile.strict_included_types`.

5.3 Lean 4 formal proof

The formal verification comprises four theorems checked in Lean 4:

1. **V1 Faithfulness**: Every token from DCA-Trie v1 is a verified KG member.
2. **V2 Faithfulness**: Stepwise expansion preserves faithfulness by induction.
3. **Monotonicity**: $P_{\text{filtered}} \subseteq P_{\text{typed}} \subseteq P_{\text{range}} \subseteq P_{\text{GCR}}$.
4. **FNR Union Bound**: As in Theorem 4.3.

3,298 verification jobs, zero errors, zero warnings.

6 Experiments

6.1 Setup

Table 1: Experimental configuration.

Setting	Value
Base model	`rmanluo/GCR-Meta-Llama-3.1-8B-Instruct`
Datasets	WebQSP (1,628 test questions, 1–2 hop) ComplexWebQuestions (up to 4 hop)
KG	Freebase ($\sim$40M entities, $\sim$350M relations)
Beam width	$k = 5$
Hop depth	$L = 2$ (WebQSP), $L = 4$ (CWQ)
Hardware	A100 40GB, 16-bit precision
Max new tokens	256 (WebQSP), 512 (CWQ)

6.2 Evaluation metrics

1. **Hits@1:** Exact match accuracy on the answer entity.

2. **F1:** Token-level entity overlap between prediction and ground truth.

3. **Structural Irrelevance Rate (SIR):** Fraction of admitted paths that are irrelevant, decomposed as:

$$\mathrm{SIR}^*_{\mathrm{type}}(q) = \frac{|\{p \in \mathcal{P}(q) : \mathrm{irrel}_{\mathrm{type}}(p, q) = 1\}|}{|\mathcal{P}(q)|}$$
$$\mathrm{SIR}^*_{\mathrm{traj}}(q) = \frac{|\{p \in \mathcal{P}(q) : \mathrm{irrel}_{\mathrm{traj}}(p, q) = 1\}|}{|\mathcal{P}(q)|}$$
$$\mathrm{SIR}^*(q) = \frac{|\{p \in \mathcal{P}(q) : \mathrm{irrel}_{\mathrm{type}}(p, q) \vee \mathrm{irrel}_{\mathrm{traj}}(p, q) = 1\}|}{|\mathcal{P}(q)|}$$

4. **False Negative Rate (FNR):** Fraction of gold-truth paths incorrectly pruned.

6.3 SIR/FNR results (WebQSP, 1,628 questions)

All SIR/FNR results are computed deterministically on the full test set; no random seeds, no threshold tuning.

Table 2: Path pruning results on the full WebQSP test set.

Metric	Value	Note
Total candidate paths (raw)	4,102,833	All DFS paths within $L = 2$
Total paths after filtering	3,509,451	
Paths pruned	593,382	
SIR (overall)	**14.5%**	Combined pruning rate
SIR_{type} (type gate)	10.6%	435,897 paths blocked
SIR_{traj} (range gate)	3.8%	157,485 paths blocked
Gold-truth paths	14,829	
Type gate FNR	**3.3%**	490 gold paths blocked
Range gate FNR	**2.9%**	424 gold paths blocked

The type gate does approximately 3× the work of the range gate (10.6% vs. 3.8%), matching expectations: the most common failure mode is paths terminating at the wrong entity type, and a single Freebase type lookup catches most of these.

Critically, both FNR values are below 5%, satisfying the requirement for practical deployment. The conservative fallback (pass when metadata is absent) ensures the oracle never catastrophically fails, unlike the cosine similarity approach at $\tau = 0.25$, which produced empty tries for 84% of queries.

6.4 Comparison with prior oracles

Table 3: Oracle comparison on WebQSP.

Method	Encoder	Threshold	GPU	Pruning
GCR (no filter)	No	No	Decode only	0%
Cosine DCA ($\tau = 0.25$)	MiniLM	Yes	Encode + decode	84% empty tries
Decomposed DCA	MiniLM	Yes	Encode + decode	TBD
Symbolic DCA (v1)	**No**	**No**	**Decode**	**14.5%**

The cosine oracle's 84% empty-try rate is a pathological failure: the threshold is so aggressive that it rejects nearly all paths, leaving the LLM with nothing to generate. The symbolic oracle has no such failure mode because it is conservative by design.

6.5 Answer type inference

The question-to-type mapper uses 19 regular expression patterns:

Table 4: Question-to-type mapping examples.

Keyword	Example question	Inferred type
who	Who directed Inception?	Person
where	Where is the Eiffel Tower?	Location
when	When was the Berlin Wall built?	Date
film/movie	What film won Best Picture?	Creative Work
country	What country is Paris in?	Country
language	What language is spoken in Brazil?	Language

6.6 Relation schema coverage

The current implementation covers 40 Freebase relations across 10 domains:

```
People:       nationality, place_of_birth, profession, ...
Film/TV:      directed_by, starring, genre, ...
Location:     capital, contains, containedby, ...
Organization: headquarters, founders, place_founded, ...
Music:        genre, composer, ...
Awards:       award_winner, honored_for, ...
Sports:       teams, sport, ...
```

Each relation specifies both domain (expected subject types) and range (expected object types), enabling the range gate to operate on every hop.

7 Related Work

7.1 Hallucination and the autoregressive mechanism

Hallucination in NLG refers to fluent yet factually unsupported content [10]. Huang et al. [9] distinguish intrinsic (contradicting context) from extrinsic (unverifiable claims) hallucination. Rates range from 15% to over 60% across tasks [10].

The root cause is architectural: at each step, the LLM computes a distribution over its vocabulary based on learned parameters and prior tokens, with no external verification [3, 23]. Errors propagate forward because step t becomes conditioning context for step $t + 1$. Scaling does not resolve this: larger models improve on frequent facts but not compositional queries [11, 17], and Xu et al. [25] prove formally that zero hallucination across all inputs is impossible for computable learners.

7.2 Knowledge graph question answering

Multi-hop KGQA requires identifying a chain $(e_0, r_1, e_1, \ldots, r_L, e_L)$ of verified KG edges [12]. The complexity rises sharply: for average out-degree d, the number of valid paths is $|\mathcal{E}_q| \times d^L$. At $d \approx 20$ and $L = 3$, this yields up to 24,000 paths, only one correct [7].

7.3 Constrained decoding for faithful generation

7.3.1 Format and entity constraints.

Grammar-Constrained Decoding [6] uses an Earley parser as the constraint oracle. Outlines [24] accelerates this with finite-state machines. PICARD [19] constrains SQL generation. GENRE [4] uses a trie of valid entity names. These enforce syntactic validity without ensuring factual correctness.

7.3.2 Knowledge graph-constrained decoding.

GCR [16] builds a KG-Trie of all KG paths within L hops and uses it as the constraint oracle. It achieves 92.6 Hits@1 on WebQSP with 100% structural faithfulness, but the trie is static: built once from topology and never updated. DoG [14] introduces step-wise trie expansion, making the oracle topologically dynamic. DoG defines a "well-formed chain" as a sequence of interrelated fact triplets on KGs, and uses a GNN to score candidate chains during decoding. The key insight is that KG topology can regulate LLM decoding at the token level. However, DoG uses topology only, without incorporating ontology metadata or question semantics. RouterKGQA [29] routes between a specialised KG model and a general LLM based on complexity, filtering at the answer level rather than the token-constraint level.

Think-on-Graph [20] treats the LLM as an agent that iteratively explores the KG using beam search. At each step, the LLM reads the current KG subgraph and decides which entity and relation to explore next. ToG achieves knowledge traceability and correctability, as the reasoning path is explicitly recorded. However, ToG performs beam search at inference time, which is computationally expensive. DCA-Trie pre-filters paths offline, enabling $O(1)$ per-token constraint checks during decoding.

7.3.3 Ontology-aware KGQA methods.

Recent work has explored using ontology metadata to improve KGQA. Ontology-Guided Reverse Thinking [8] constructs reasoning paths from answer types back to conditions using the KG ontology, then uses these paths to guide knowledge retrieval. ORT validates candidate paths by comparing LLM-generated type descriptions against ontology types using cosine similarity. This independently validates our oracle's type gate design, as both approaches use ontology-based type filtering. The key difference is that ORT uses ontology for *path construction*, while DCA-Trie uses it for *path filtering*. These are complementary approaches that can be combined: ORT constructs candidate paths, then DCA-Trie filters them using symbolic gates.

7.4 Retrieval-augmented generation

RAG [13] grounds LLM outputs in external knowledge. KG-augmented variants retrieve subgraphs as context [7, 26]. Self-RAG [1] adds reflection tokens; IRCoT [22] interleaves retrieval with CoT; GraphRAG [5] uses community-structured retrieval.

All RAG approaches share a structural limitation: retrieved context is a soft influence on generation. The retriever determines what the LLM *sees*, not what it is *permitted to generate*. Luo et al. [16] find that even with a KG directly available, retrieval-augmented approaches hallucinate in approximately one-third of cases.

7.5 Three unresolved gaps

Table 5: Comparison of DCA-Trie with prior work.

Method	Faithful	Semantic	Encoder	Token-level	Ontology
GCR	100%	No	No	Yes	No
DoG	100%	No	No	Yes	No
ToG	Probabilistic	Soft	No	No	No
ORT	Probabilistic	Explicit	Yes	No	Yes
RouterKGQA	Probabilistic	Yes (answer)	Yes	No	No
RAG / GNN-RAG	No	Soft	Yes	No	No
Cosine DCA	100%	Implicit	Yes	Yes	Implicit
Symbolic DCA	**100%**	**Explicit**	**No**	**Yes**	**Yes**

Gap 1: The permissiveness problem. Current KG-constrained decoding builds oracles from graph structure only, ignoring question semantics and generation state. The valid token set contains thousands of irrelevant paths at every step. Even ToG [20], which uses beam search to explore the KG, does not filter the token set at each step.

Gap 2: The static-dynamic tradeoff. GCR's static trie is efficient but semantically blind. DoG's dynamic expansion adds no semantic filtering. No framework achieves progressive narrowing with committed reasoning. ORT [8] uses ontology for path construction, not filtering, and requires an encoder for cosine similarity.

Gap 3: No constraint quality metric. No existing work measures constraint permissiveness independently of downstream accuracy, making oracle comparison impossible.

8 Conclusion

We have presented DCA-Trie, a framework for incorporating KG ontology metadata into constrained decoding for KGQA. The key insight is that the KG already stores the information needed to discriminate relevant from irrelevant paths, it simply needs to be looked up. Our symbolic oracle, consisting of two $O(1)$ set-containment checks, prunes 14.5% of candidate paths on WebQSP while retaining 96.7% of gold-truth paths.

The monotonicity guarantee (Theorem 4.2) ensures no false positives relative to GCR, and the union-bound FNR (Theorem 4.3) provides a formal recall bound. The Lean 4 proof (3,298 jobs, zero errors) provides machine-checked assurance.

Future work. The 40-relation schema can be extended to Freebase's full relation set (∼24,000). The question-to-type mapper (19 regex patterns) could be learned from the KG's type hierarchy. DCA-Trie can be combined with retrieval augmentation for systems needing both token-level guarantees and broader graph awareness.

Reproducibility. All code is available at `https://github.com/bernard/graph-constrained-reasoning`. Experiments ran on a single A100 40GB GPU with 16-bit precision. The Lean 4 proof is in `proof/formal_proof/`.

References

[1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*, 2024.

[2] Kurt Bollacker, Colin Evans, Praveen Paritosh, Tim Sturge, and Jamie Taylor. Freebase: A collaboratively created graph database for structuring human knowledge. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pages 1247–1250, 2008.

[3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.

[4] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. Autoregressive entity retrieval. In *International Conference on Learning Representations*, 2021.

[5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[6] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL `https://openreview.net/forum?id=KkHY1WGDII`.

[7] Gaole He, Yunshi Lan, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. Improving multi-hop knowledge base question answering by learning intermediate supervision signals. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining (WSDM)*, pages 553–561. ACM, 2021.

[8] Chen Huang, Xiusi Chen, Junzhou Zhang, Heng Li, and Yu Zheng. Think reverse and reason: Ontology-guided reverse thinking for knowledge graph reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17824–17838, 2025.

[9] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.

[10] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 55(12), March 2023. ISSN 0360-0300. doi: 10.1145/3571730.

[11] Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. Large language models struggle to learn long-tail knowledge. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 15696–15707. PMLR, 2023.

[12] Yunshi Lan, Gaole He, Jinhao Jiang, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. Complex knowledge base question answering: A survey. *arXiv preprint arXiv:2108.06688*, 2022.

[13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.

[14] Kun Li, Tianhua Zhang, Xixin Wu, Hongyin Luo, James R. Glass, and Helen M. Meng. Decoding on graphs: Faithful and sound reasoning on knowledge graphs through generation of well-formed chains. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24349–24364. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1186. URL `http://dx.doi.org/10.18653/v1/2025.acl-long.1186`.

[15] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pages 3214–3252. Association for Computational Linguistics, 2022.

[16] Linhao Luo, Zicheng Zhao, Gholamreza Haffari, Yuan-Fang Li, Chen Gong, and Shirui Pan. Graph-constrained reasoning: Faithful reasoning on knowledge graphs with large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 41540–41565. PMLR, 2025. URL `https://proceedings.mlr.press/v267/luo25t.html`.

[17] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 9802–9822. Association for Computational Linguistics, 2023.

[18] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019.

[19] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021.

[20] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *International Conference on Learning Representations*, 2024.

[21] Alon Talmor and Jonathan Berant. The web as a knowledge-base for answering complex questions. In *Proceedings of the 2018 Conference of the North American Chapter of the Asso-*

ciation for Computational Linguistics: Human Language Technologies (NAACL-HLT), pages 641–651. Association for Computational Linguistics, 2018.

[22] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 10014–10037. Association for Computational Linguistics, 2023.

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[24] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023.

[25] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. Hallucination is inevitable: An innate limitation of large language models, 2025.

[26] Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. QA-GNN: Reasoning with language models and knowledge graphs for question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 535–546. Association for Computational Linguistics, 2021.

[27] Susumu Yata et al. marisa-trie: Matching algorithm with recursively implemented storage. `https://github.com/s-yata/marisa-trie`, 2024.

[28] Wen-tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 201–206. Association for Computational Linguistics, 2016.

[29] Bo Yuan, Hexuan Deng, Xuebo Liu, and Min Zhang. Routerkgqa: Specialized–general model routing for constraint-aware knowledge graph question answering, 2026.